

Deep Learning — Week 8 Notes

Unsupervised Pre-training · Better Activations (ReLU) · Weight Initialization (Xavier / He) · Batch Normalization

NPTEL / IIT Ropar · CS7015 Lecture 9 · Exam-focused revision sheet

★ Includes full worked answer key (all 20)

1 · Why Deep Nets Are Hard to Train

Saturating sigmoids kill the gradient

A sigmoid **saturates** when its output is near 0 or 1 — there its slope is almost flat, so the gradient flowing back is tiny. Stacked over many layers, gradients **vanish** and early layers barely learn.

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \rightarrow \approx 0 \text{ when saturated}$$

- Sigmoids are not zero-centered and are computationally expensive (exponential).
- Deep sigmoid networks train slowly because **saturated units produce very small gradients**.
- A sigmoid at the **output** of a binary classifier is still fine — it **maps the score to a probability in (0,1)**.

2 · Unsupervised (Greedy Layer-wise) Pre-training

Build the network one layer at a time

Train each hidden layer as an **unsupervised reconstruction** problem (like an autoencoder), **freeze** its weights, then train the next layer on the previous layer's representation. After all layers are pre-trained, **add the output layer and fine-tune** the whole network with the supervised labels.

- Goal: give each layer a **useful initialization before supervised fine-tuning**.
- During pre-training of a later layer, the already-trained layer below is **kept fixed**.
- The pre-training objective is **unsupervised** — it uses the input data **without the task labels**.
- It reduces the **difficulty of training early layers in a very deep network**.

Case Study 3 — deep warehouse fault detector, weak deep-layer representations:

goal = **useful initialization before fine-tuning** · first layer stays **fixed** while the next is trained · first objective is unsupervised because it **uses inputs without the abnormal-condition labels** · after all layers pre-trained, **the full model is fine-tuned on the supervised task** · reduces **difficulty of training early layers in a very deep network**.

3 · Better Activation Functions (ReLU)

ReLU: $\max(0, x)$

$$\text{ReLU}(x) = \max(0, x) \cdot \text{derivative} = 1 \text{ if } x > 0, \text{ else } 0$$

- **Does not saturate** in the positive region → keeps a useful gradient, trains faster than sigmoid.
- Cheap to compute. Downside: **dead neurons** (a unit stuck at 0 for all inputs gets zero gradient forever).
- Variants: **Leaky ReLU** (small negative slope), **Maxout**, **ELU** — designed to avoid dead units.

Case Study 1 — sigmoid vs ReLU on insurance features:

sigmoid slows deep training because units **saturate and give very small gradients** · $\text{ReLU}(-4) = 0$ · $\text{ReLU}(3) = 3$ · ReLU helps positive pre-activations because it **avoids saturation and keeps a useful gradient** · output sigmoid kept because it **maps the final score to a probability between 0 and 1**.

4 · Weight Initialization (Xavier / He)

Scale the initial variance to the layer size

If all weights start equal, neurons stay identical forever (**symmetry never breaks**). If variance is too large or too small, activations **explode or vanish**. Good init sets the variance based on the number of inputs n .

$$\text{Xavier: } \text{Var}(w) = 1/n \quad \cdot \quad \text{He: } \text{Var}(w) = 2/n \quad (n = \text{input connections})$$

- **Xavier (Glorot)**: good for tanh / sigmoid layers.
- **He**: uses the factor 2 — designed for **ReLU** layers (compensates for ReLU zeroing half the activations).

Case Study 2 — 7 ReLU hidden layers, unstable with variance 1, hidden layer has $n = 200$ inputs:

poor init can **cause activations/gradients to explode or vanish** · Xavier variance = $1/200 = 0.005$ · He variance = $2/200 = 0.010$
· He is more suitable here because **the hidden layers use ReLU activations** · improved stability shows as **loss decreasing more smoothly across epochs**.

5 · Batch Normalization

Normalize each layer's pre-activations per mini-batch

As training proceeds, the distribution of a layer's inputs keeps shifting (**internal covariate shift**), which slows learning. Batch norm normalizes each pre-activation over the mini-batch to zero mean / unit variance, then applies a **learnable scale γ and shift β** .

$$\hat{x} = (x - \mu_{\text{batch}}) / \sqrt{(\sigma^2_{\text{batch}} + \epsilon)} \rightarrow y = \gamma \hat{x} + \beta$$

- Placed **between the linear transformation and the ReLU activation**.
- The learnable γ, β let the network **recover any representation it needs** — normalization is not forced to stay fixed.
- Reduces the shifting of layer-input distributions → faster, more stable training.

Case Study 4 — mini-batch pre-activations {1, 3, 5, 7}:

batch norm addresses **shifting layer-input distributions during training** · mean = $(1+3+5+7)/4 = 4$ · variance = $[(1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2]/4 = (9+1+1+9)/4 = 5$ · placed **between the linear transformation and ReLU** · γ/β useful because they **let the network choose an appropriate normalized representation**.

★ FULL ANSWER KEY — Week 8 (assignment not submitted)

Q	Answer	Q	Answer
1	They saturate → very small gradients	11	Useful init before supervised fine-tuning
2	0 (ReLU(-4))	12	It is kept fixed while next layer trains
3	3 (ReLU(3))	13	Uses input without the condition labels
4	Avoids saturation, keeps a useful gradient	14	Full model fine-tuned on supervised task
5	Maps final score to probability (0,1)	15	Difficulty of training early deep layers
6	Activations/gradients explode or vanish	16	Reduces shifting layer-input distributions
7	0.005 (Xavier = $1/200$)	17	4 (mean of 1,3,5,7)
8	0.010 (He = $2/200$)	18	5 (batch variance)
9	Hidden layers use ReLU activations	19	Between linear transform and ReLU
10	Loss decreases more smoothly	20	Choose an appropriate normalized rep.